

OS Assignment 5: IPC Assignment Report

Assignment 1-2: Shared Memory Buffer With Semaphore

이 과제에서는 Assignment 1-1의 공유 메모리 버퍼를 기반으로 POSIX 세마포어 동기화를 추가했습니다. 세마포어는 단일 슬롯 버퍼의 임계 영역을 보호하여 한 번에 하나의 프로세스만 공유 버퍼 상태를 읽거나 쓸 수 있도록 합니다.

s_wait()와 s_quit()를 사용한 위치

- produce()
 - s_wait()를 호출하여 is_empty 상태를 확인하고 쓰기 작업을 시작하기 전에 임계 영역을 잠급니다.
 - s_quit()를 호출하여 message.sender_id, message.data, account_id를 모두 기록하고 is_empty를 0으로 변경한 뒤 임계 영역을 해제합니다.
- consume()
 - s_wait()를 호출하여 is_empty 상태를 확인하고 메시지를 읽기 전까지 임계 영역을 잠급니다.
 - s_quit()를 호출하여 메시지를 읽고 is_empty를 1로 업데이트한 뒤 임계 영역을 해제합니다.

왜 이 위치가 임계 영역인지

공유 메모리 버퍼는 생산자와 소비자가 동시에 접근하는 공용 자원입니다. 다음 작업은 모두 임계 영역에 속합니다.

- is_empty 읽기/쓰기
- message.sender_id, message.data, account_id 쓰기
- 버퍼 상태를 is_empty로 변경하는 작업

동기화가 없으면 생산자와 소비자가 같은 시간에 버퍼를 읽고 쓰면서 데이터가 손상되거나 상태가 잘못 해석될 수 있습니다.

Assignment 1-1에서 발생할 수 있는 Race condition

Assignment 1-1에서는 생산자와 소비자가 세마포어 없이 버퍼를 직접 접근합니다. 따라서 다음과 같은 문제점이 생깁니다.

- 생산자가 메시지를 쓰는 도중 소비자가 동일한 버퍼를 읽을 수 있음
- 소비자가 is_empty를 확인한 뒤 다른 프로세스가 버퍼를 변경하여 잘못된 데이터를 읽을 수 있음
- 생산자가 소비자가 메시지를 읽기 전에 버퍼를 덮어쓸 수 있음

이로 인해 소비자는 불완전하거나 일관성 없는 데이터를 읽을 수 있고, 버퍼 상태는 잘못된 값으로 바뀔 수 있습니다.

세마포어 및 정리 동작

- init_sem()에서 sem_open()을 사용하여 초기값 1로 POSIX 명명 세마포어를 생성/연결합니다.

- s_wait()는 sem_wait()으로 임계 영역 진입을 직렬화합니다.
- s_quit()는 sem_post()로 임계 영역을 해제합니다.
- destroy_sem()는 sem_close()로 세마포어를 닫고 sem_unlink()로 제거합니다.
- 공유 메모리 정리는 munmap(), close(), shm_unlink()로 수행합니다.

스크린샷 항목

1. 세마포어 동기화 추가 전 실행 화면

```

silver@silver-TFX5470H$ ./consumer &
[1] 98089
silver@silver-TFX5470H$ init buffer
sender_id : 98124
account_id : 1001
balance : 500

sender_id : 98137
account_id : 1002
balance : 200

silver@silver-TFX5470H$ ./producer 1001 500
attach buffer
produce message
silver@silver-TFX5470H$ ./producer 1002 200
attach buffer
produce message
silver@silver-TFX5470H$

```

2. 세마포어 동기화 추가 후 실행 화면 (안정적으로 동작하는 상태)

```

silver@silver-TFX5470H$ ./consumer &
[1] 97033
silver@silver-TFX5470H$ init buffer
init semaphore : /ipc_assignment_message_sem
sender_id : 97134
account_id : 1001
balance : 500

sender_id : 97169
account_id : 1002
balance : 200

silver@silver-TFX5470H$ ./producer 1001 500
attach buffer
produce message
silver@silver-TFX5470H$ ./producer 1002 200
attach buffer
produce message
silver@silver-TFX5470H$

```

세마포어(semaphore)를 사용하지 않은 경우에도 실험 실행 중 명확한 오류가 발생하지 않을 수 있습니다. 이는 운영체제의 스케줄링 결과 생산자와 소비자가 우연히 충돌하지 않는 순서로 실행되었기 때문이며, 프로그램이 항상 올바르게 동작한다는 의미는 아닙니다.

실행 예시

```

./consumer &
./producer 1001 500
./producer 1002 200
./destroy

```

Assignment 2: Named Pipe Client/Server

why two named pipes are used

Named pipe는 기본적으로 한 방향 통신에 사용되고, 과제에선 양방향 통신이 필요하므로 두 개의 named pipe가 필요합니다.

the role of the client-to-server pipe

client_to_server는 클라이언트가 서버에게 정수 값을 보내는 파이프입니다. 클라이언트는 이 파이프에 값을 write()하고, 서버는 이 파이프에서 값을 read()합니다.

the role of the client-to-server pipe

server_to_client는 서버가 계산 결과를 클라이언트에게 보내는 파이프이다.

서버는 클라이언트에게 받은 정수를 제공한 뒤 이 파이프에 write()하고, 클라이언트는 결과를 read()해서 출력한다.

the role of the server-to-client pipe

server_to_client는 서버가 계산 결과를 클라이언트에게 보내는 파이프입니다. 서버는 클라이언트에게 받은 정수를 제공한 뒤 이 파이프에 write()하고, 클라이언트는 결과를 read()해서 출력합니다.

client.c에 추가한 코드 설명

```
/*-----*/
/* TODO 1 : init receive_fd and send_fd */

// client_to_server pipe(NP_SEND)를 쓰기 전용(O_WRONLY 모드)으로 열어서 서
버에게 값을 보낼 준비를 한다.
send_fd = open_fifo(NP_SEND, O_WRONLY);
if (send_fd == -1) {
    perror("open client_to_server");
    return 1;
}

// server_to_client pipe를 읽기 전용으로 열어서 서버의 계산 결과를 받을 준비를
한다.
receive_fd = open_fifo(NP_RECEIVE, O_RDONLY);
if (receive_fd == -1) { //pipe 열기 실패 시
    perror("open server_to_client"); //에러 메시지 출력
    close(send_fd);
    return 1;
}

/* TODO 1 : END */
/*-----*/
```

```
/*-----*/
/* TODO 2 : send msg and receive msg */

// send_msg에는 사용자가 입력한 정수 값이 문자열 형태로 들어 있다.
// strlen(send_msg)는 문자열 길이만 계산하고, +1은 문자열 끝의 NULL 문자('\0')까지
함께 보내기 위한 것이다.
// 즉, 서버가 pipe에서 읽었을 때 문자열의 끝을 정확히 알 수 있게 한다.
```

```

if (write(send_fd, send_msg, strlen(send_msg) + 1) == -1) {
// write 반환 값이 -1 일 경우 쓰기 실패. 에러 메시지 출력
    perror("write");
    break;
}

// receive_msg 버퍼를 0으로 초기화
// 이전에 읽었던 데이터가 버퍼에 남아 있으면 새로 받은 결과와 섞일 수 있기 때문이다.
memset(receive_msg, 0, BUFFER_SIZE);

// receive_fd를 통해 서버가 보낸 결과 문자열을 읽는다.
// BUFFER_SIZE - 1만큼만 읽는 이유는 마지막 공간을 NULL 문자('\0')를 위해 남겨두기
// 위해서이다.
if (read(receive_fd, receive_msg, BUFFER_SIZE - 1) <= 0) {
// read()의 반환값이 0 이하이면 읽기 실패 또는 pipe 연결 종료로 판단
    perror("read");
    break;
}

/* TODO 2 : END */
/*-----*/

```

server.c에 추가한 코드 설명

```

/*-----*/
/* TODO 3 : make pipes and init fds */
/* (1) make NP_RECEIVE pipe */
/* (2) make NP_SEND pipe */
/* (3) init receive_fd and send_fd */

/*
 * NP_RECEIVE: server가 client로부터 데이터를 받는 pipe
 * NP_SEND : server가 client에게 결과를 보내는 pipe
 */

/* 기존에 남아 있을 수 있는 receive pipe를 삭제한다.
 * 이전 실행에서 pipe 파일이 남아 있으면 mkfifo()가 실패할 수 있기 때문이다.
 */
unlink(NP_RECEIVE);

/* client -> server 방향의 named pipe를 생성한다.
 * 이 pipe를 통해 server는 client가 보낸 입력값을 받는다.
 * 0666은 읽기/쓰기 권한을 의미한다.
 */
if (mkfifo(NP_RECEIVE, 0666) == -1) {
    perror("mkfifo client_to_server");
}

```

```

    return 1;
}

/* 기존에 남아 있을 수 있는 send pipe를 삭제한다. */
unlink(NP_SEND);

/* server -> client 방향의 named pipe를 생성한다.
 * 이 pipe를 통해 server는 계산 결과를 client에게 보낸다.
 */
if (mkfifo(NP_SEND, 0666) == -1) {
    perror("mkfifo server_to_client");

    /* 두 번째 pipe 생성에 실패했으므로,
     * 앞에서 만든 receive pipe도 삭제하고 종료한다.
     */
    unlink(NP_RECEIVE);
    return 1;
}

/* NP_RECEIVE pipe를 읽기 전용으로 연다.
 * server는 이 fd를 사용해서 client가 보낸 메시지를 read()로 읽는다.
 */
receive_fd = open(NP_RECEIVE, O_RDONLY);
if (receive_fd == -1) {
    perror("open client_to_server");

    /* pipe 열기에 실패했으므로 생성했던 pipe 파일들을 정리한다. */
    unlink(NP_RECEIVE);
    unlink(NP_SEND);
    return 1;
}

/* NP_SEND pipe를 쓰기 전용으로 연다.
 * server는 이 fd를 사용해서 계산 결과를 write()로 client에게 보낸다.
 */
send_fd = open(NP_SEND, O_WRONLY);
if (send_fd == -1) {
    perror("open server_to_client");

    /* receive_fd는 이미 열려 있으므로 닫아 주고,
     * 생성했던 pipe 파일들도 삭제한다.
     */
    close(receive_fd);
    unlink(NP_RECEIVE);
    unlink(NP_SEND);
    return 1;
}

```

```

/* TODO 3 : END */
/*-----*/
...

```

```

/*-----*/
/* TODO 4 : read msg */

/* receive_msg 버퍼를 0으로 초기화한다.
 * 이전에 읽은 데이터가 남아 있으면 새로 읽은 문자열과 섞일 수 있기 때문이다.
 */
memset(receive_msg, 0, BUFFER_SIZE);

/* NP_RECEIVE pipe에서 client가 보낸 메시지를 읽는다.
 * server는 이 값을 읽은 뒤 정수로 변환하여 계산에 사용한다.
 * BUFFER_SIZE - 1만큼만 읽는 이유는 문자열 끝의 NULL 문자('\0') 공간을 남겨두기
   위해서이다.
 */
bytes_read = read(receive_fd, receive_msg, BUFFER_SIZE - 1);
if (bytes_read <= 0) {
    break;
}

/* TODO 4 : END */
/*-----*/

```

```

/*-----*/
/* TODO 5 : write msg */

/* send_msg에는 server가 계산한 결과가 문자열 형태로 저장되어 있다.
 * 이 문자열을 NP_SEND pipe에 써서 client에게 전달한다.
 * strlen(send_msg) + 1은 문자열 끝의 NULL 문자('\0')까지 함께 보내기 위한 것이
   다.
 */
if (write(send_fd, send_msg, strlen(send_msg) + 1) == -1) {
    perror("write");
    break;
}

/* TODO 5 : END */
/*-----*/

```

```
silver@silver-TFX5470H$ ./server
server : receive 1

server : send 1
server : receive 2

server : send 4
server : receive 3

server : send 9
server : receive 4

server : send 16
server : receive 5

server : send 25
server : receive 6

server : send 36
server : receive 7

server : send 49
server : receive 8

server : send 64
server : receive 9

server : send 81
silver@silver-TFX5470H$
```

```
silver@silver-TFX5470H$ ./client
client : send 1
client : receive 1

client : send 2
client : receive 4

client : send 3
client : receive 9

client : send 4
client : receive 16

client : send 5
client : receive 25

client : send 6
client : receive 36

client : send 7
client : receive 49

client : send 8
client : receive 64

client : send 9
client : receive 81

silver@silver-TFX5470H$
```